02 — Agent Engineering

This document is about the scaffold: agent graph, prompt, tool policy, budgets, context management, and the public configurations that have scored. It assumes you have read `01-harness-and-evaluation.md` for the exact contract.

The central public finding: **a prompt-only 31B notebook claims 0.24 public** (S36). Treat the exact score as author-reported until you reproduce it; the official top public score is 0.24 (S4). The 0.24 config is a single `LlmAgent` with a disciplined prompt, 8 minutes/task, 28 tool calls, 8192 max output, 4096 thinking, and all nine tools declared but three graph tools explicitly discouraged. Before building a complex multi-agent system or a LoRA, reproduce that baseline.

---

1. Architecture choices

1.1 Single `LlmAgent` (recommended default)

A single agent with:

- `run_command`, `read_file`, `edit_file`, `write_file`, `get_status`, `submit_patch`;
- optionally graph tools, if they help on your tasks;
- a short, explicit workflow prompt;
- one budget.

This is the simplest and the strongest public baseline (S36–S38). It avoids handoff overhead, context duplication, and per-agent budget starvation. The 31B model is capable enough to inspect, edit, and verify when the prompt forces a plan.

Use this for your first submission and for any task set where context is not the bottleneck.

1.2 Coder + read-only explorer (`agent_tool`)

A root coder with a read-only analyzer exposed as an `agent_tool`:

```
tools:
  - run_command
  - read_file
  - edit_file
  - write_file
  - get_status
  - submit_patch
  - agent_tool:
      config_path: sub_agents/code_analyzer.yaml
      skip_summarization: true
```

The analyzer can use `run_command`, `read_file`, and graph tools; the root does the editing. `skip_summarization: true` keeps the analyzer's intermediate outputs out of the root context (S5:626). Set `include_contents: none` on the analyzer if it should not see the root's history.

This can help when exploration consumes the 32k window. It also adds latency and a second model call path. Public evidence is mixed: a 12B coder + 12B reviewer solved 3/31 vs 4/31 for the coder alone (S34). Do not assume multi-agent is better; measure it.

1.3 Sequential pipeline

`SequentialAgent` with analyze → patch → verify stages (S5:108). This is tempting but risky:

- There is **one** tool-call/time/turn budget for the whole task; there is no per-agent budget in the schema (S28, S5:499-509).
- The first stage can consume the budget and starve the editor.
- Handoffs add context and latency.

If you use a pipeline, give the analyzer a hard, small scope (e.g., "return a file/line/root-cause hypothesis in ≤10 calls") and make the coder stage the only one that can edit. Measure stage-level time; a first stage can consume the whole task budget.

1.4 Parallel branches

`ParallelAgent` runs sub-agents concurrently in isolated branches (S5:110). This can explore multiple hypotheses, but it multiplies model calls and time. With a 12h global

budget and a 32k context, parallel exploration is usually a luxury. Use it only for independent, cheap analyses (e.g., “find tests” and “find callers” in parallel), not for full coding attempts.

1.5 LoopAgent

`LoopAgent` repeats sub-agents up to 500 iterations (S5:112). Avoid it for the main coding loop; the harness already drives a multi-turn loop and has nudges. A `LoopAgent` can burn the budget without an exit condition; `exit_loop` is not among the nine tools (S5:341). If you use it, keep `max_iterations` small and put a hard stop in the prompt.

---

2. The public prompt-only configuration (author-reported 0.24)

The public prompt-only notebook (author-reported 0.24; S36) is the best starting point. It copies `agent.yaml`, `configs/sampling.yaml`, and `eval_config.yaml` byte-for-byte from the public 0.22 notebook (S37), and changes only the prompt.

2.1 Files

`agent.yaml`:

```
name: swe_agent
model: gemma-4-31b-it-qat-w4a16-ct
instruction: !include prompts/system.md
tools:
  - run_command
  - read_file
  - edit_file
  - write_file
  - get_status
  - submit_patch
  - get_code_neighbors
  - search_similar_code
  - get_code_subgraph
generate_content_config: !include configs/sampling.yaml
```

`configs/sampling.yaml`:

```
  temperature: 0.2
  top_p: 0.95
  max_output_tokens: 8192
  thinking_config:
    thinking_budget: 4096
    include_thoughts: true
```

`eval_config.yaml`:

```
  evaluation:
    timeout_seconds: 240
    max_tool_calls: 28
    max_time_minutes: 8
    max_turns: 80
```

These are exact values from the public notebook (S36). They are a **reference configuration**, not the default recommendation.

Recommended competition bundle: 5 min/task, 24 calls, 240 s command timeout, 8192 output, thinking 2048. The public 8-min config's measured mean is ~5.4 min (S34); 120 × (5.4 + setup) can approach or exceed 12 h (S29). Use 8 min only if your measured `120 × (agent_p50 + setup_p50)` is comfortably below 9.5 h. Keep the 8-min bundle as a known-good fallback.

Timeout correction: `evaluation.timeout_seconds` maps to the Phase-1 per-command timeout only. The scorer's Phase-2 pytest timeout is fixed at 300 s and is not configurable via `eval_config.yaml` (S5:500,506). The 240 s value is an agent-command setting, not a scoring guarantee. Do not shorten or lengthen it expecting to change test scoring.

2.2 Prompt

The public prompt-only prompt is long but effective. Its key rules:

- **Hard limits:** 28 calls / 8 minutes; an edit that exists counts even without `submit_patch`; an unedited tree scores zero.
- **Never edit tests** (`test_*.py`, `*_test.py`, anything under `tests/`, `pytest.ini`, `conftest.py`).
- **Scratch files only in `/tmp`**; anything under `/workspace` enters the patch.
- **Never run a full test suite**; name one test file and use `-k`.

- **Never look outside `/workspace`.**
- **Never use the three graph tools** ("They return nothing useful").
- **Call plan:** calls 1–8 locate/read; calls 9–12 edit; **must edit by call 12**; calls 13–20 verify/correct; call 20+ no new exploration; finish with `git status --short && git diff` then `submit_patch`.
- **Procedure:** extract exact names/messages/types/edge cases; locate with `git grep`/`read_file`; edit with `edit_file`; if edit fails twice, use one inline Python replace with an assert; fix the cause, not the symptom; apply to parallel implementations; use exact exception/message; verify at most 2 calls; final check; submit; confirm `patch_size > 0` and `files_changed > 0`.
- **Decision rules:** ambiguous issue → most literal reading; several locations → where the wrong behavior is produced; search empty → one different string then edit best candidate; stuck → different smaller change; never conclude nothing needs to change.

The full prompt is in the public notebook (S36). Use it as a template, not as a magic incantation. The important properties are: a hard edit deadline, source-only edits, patch hygiene, and a final diff/submit sequence.

2.3 Why this config works

The public 0.18 analysis gives the rationale (S38):

- **8 minutes:** in 129 local runs, only about half of the fixes Gemma eventually finds are in the file after 4 minutes of scorer-speed time.
- **28 calls:** runs still exploring after many calls almost never succeed; a hard cap stops them and pays for longer time.
- **Edit by call 12:** runs with no source edit by call 12 fail 87% of the time.
- **8192 output:** vLLM refuses requests where prompt + max output >32,768.
- **240 s timeout:** a Phase-1 command setting that lets long agent tests finish; it does not change the scorer's fixed 300 s Phase-2 timeout (S5:506).
- **Graph tools discouraged:** exact fully-qualified IDs only; usually return nothing.

These are community measurements on public tasks, not official scorer guarantees. They are a good prior, not a law.

2.4 What to change first

If you start from this config, change one variable at a time:

1. **Budget:** 5/6/8 minutes; 24/28/32 calls. Measure mean/p95 time and patch rate.
2. **Thinking:** 0/512/2048/4096. More thinking can reduce loops but costs time; uncapped thinking + 5 min timed out 15/22 in one study (S39).
3. **Output cap:** 8192 vs 12288/16384. Higher can truncate less but leaves less prompt room; vLLM rejects prompt + output >32k (S38).
4. **Graph tools:** declared-but-discouraged vs removed vs used. Public audits say weak; your private tasks may differ.
5. **Prompt edit deadline:** call 10/12/14. Earlier edits reduce pure reading but may increase wrong edits.
6. **Sub-agent:** add a read-only explorer only if context is the bottleneck.
7. **Adapter:** only after the prompt-only baseline is stable and scored.

Do not change model, prompt, budget, and adapter at once. You will not know what caused the score change.

---

3. Budget allocation

3.1 The global constraint

The 12h limit is global, not per task (S1). With ~120 tasks, the mean task time must be low enough to leave margin. A rough planning model:

$$T_{\text{total}} \approx T_{\text{startup}} + \sum_{i=1}^{N} (T_{\text{setup},i} + T_{\text{agent},i})$$

The per-task cap is only a safety valve. If your mean agent time is 5.5 min and N=120, agent time alone is 11h; setup/startup pushes you over. If your mean is 4.5 min, agent time is 9h; there is margin. The public 0.18 notebook says its caps kept a scored run under 12h (S38), but the public prompt-only config uses 8 min and reportedly scored; tasks must finish early on average.

Measure:

- `p50`, `p90`, `p95` **agent** task time on a 20–40 task holdout.
- `setup_p50`/`setup_p95` (container setup, wheel install, snapshot extraction) per task. The harness excludes setup from the agent timer but the global 12 h includes it (S5:511, S1).
- Projected total `startup + N × (agent_p50 + setup_p50)` with N≈120.

- Percentage of tasks that hit the per-task cap.
- Percentage of tasks with a non-empty patch before the cap.
- A100→L4 derating: measure decode tokens/s on your A100 stack and compare to the public 4×L4 measurement (~33–35 tok/s for 31B; S34). Multiply the model-time component by the ratio before projecting; or use the Kaggle L4x4 rehearsal as the authoritative timing check.

If >20% of tasks hit the cap and produce no patch, the cap is too tight or the prompt is too exploratory. If the projected total is >10.5h, reduce the cap or the thinking budget.

3.2 Tool-call budget

- 10 calls (sample) is too low; the starter run hit it without an edit (S6:491-516).
- 28 calls is the public prompt-only baseline (S36).
- 32–40 calls can help hard tasks but increase loops/time; a public comparison found 36 calls worse than 28 locally (S39).
- 50–100 calls is usually too much for a 5–8 min cap; the model spends the budget exploring.

Use `get_status()` (free) and heed the `budget_warning` after 20 calls/≤10 remaining (S5:348-350). Put a rule in the prompt: “when the budget warning appears, stop exploring; make the fix or submit.”

3.3 Thinking budget

- 0: fastest, but more malformed tool calls and repeated loops (S39).
- 512: a good reliability/cost trade-off in one 22-task study; 7/22 vs 5/22 thinking off (S39).
- 4096: public prompt-only baseline; more deliberate, slower (S36).
- > 4096/uncapped: can time out; uncapped + 5 min timed out 15/22 and scored 0.05 (S39).

Start at **2048** for the primary 5-min/24-call bundle. The public prompt-only reference uses 4096; treat it as an A/B, not the default. If mean time is too high, try 1024/512. If loops/malformed calls are high, increase slightly or improve the prompt’s “never repeat a failed call” rule.

3.4 Output cap

`max_output_tokens` is the requested output allowance, not the actual output length. It counts against the 32k context along with the prompt. A cap of 8192 leaves room for long prompts; 16384 can cause vLLM to reject requests when prompt + output >32,768 (S38). Use 8192 as the default; raise only if you see frequent truncation and your prompts are short.

---

4. Prompt design

4.1 What the prompt must do

The harness already provides the problem statement, hints, budget, environment rules, and workspace layout (S5:282-319). Your system prompt should add:

1. **A hard edit deadline.** "You must edit a source file by call N."
2. **Source-only discipline.** "Never edit tests or runner config; they are stripped."
3. **Patch hygiene.** "Scratch files in `/tmp`; check `git status --short && git diff`; submit last."
4. **A localization strategy.** "Use exact paths/symbols from the issue; batch independent searches; read small regions; find where the wrong behavior is produced."
5. **An edit strategy.** "Smallest unique `old_string`; if edit fails twice, one inline Python replace with an assert; fix the cause."
6. **A verification strategy.** "At most two targeted test calls; ignore pre-existing fixture/import errors; don't repair the environment."
7. **A no-loop rule.** "Never repeat an identical failed call; change the arguments or approach."
8. **A finalization rule.** "When the budget warning appears, stop exploring; fix or submit."
9. **A no-leak rule.** "Do not search outside `/workspace` for fixed copies."
10. **A no-test-edit rule.** Explicitly list the protected patterns.

4.2 Prompt length and compaction

The context window is 32k. Compaction triggers at 14,336 tokens (S5:518-521). A few-thousand-character prompt is roughly ~1k tokens [ESTIMATE]; that is fine. A 20k-character prompt leaves little room and can be partially dropped. Keep the prompt

concise; put the most important rules near the top and repeat the finalization rule near the bottom.

The public prompt-only prompt is a few thousand characters; keep yours under ~4k. That is a good size. Do not write a 10k-word “comprehensive” prompt; it will hurt.

4.3 Tool-call grammar

Gemma 4 emits tool calls with special tokens, not JSON (S39). Malformed quoting can swallow the next argument; the parser error may only say a parameter is missing. The model may repeat the same malformed call. Mitigations:

- Keep string arguments short and simple.
- Split large edits into smaller `edit_file` calls.
- Use `write_file` only for genuinely new files.
- If `edit_file` fails twice, switch to one inline Python replace via `run_command`, with an assert that the old text occurs exactly once.
- Tell the model: “If a tool call fails, change the arguments; never resend the identical call.”
- Use a bounded thinking budget; thinking reduces malformed calls but costs time (S39).

4.4 A prompt skeleton

Use the public prompt-only prompt as the base (S36). If you write your own, this is the minimum:

```
You are an autonomous software engineer. Fix the issue in /workspace.

HARD LIMITS
- 24 tool calls and 5 minutes (use 28/8 only for the public reference). An
edit that exists counts; an unedited tree scores zero.
- Never edit tests, conftest.py, pytest.ini, pyproject.toml, or other runner
config.
- Scratch files go in /tmp only.
- Never run a full test suite. Name one test file and use -k.
- Never look outside /workspace.
- Never repeat an identical failed tool call.

CALL PLAN
- Calls 1-8: locate and read. Calls 9-12: edit. You MUST edit a source file
by call 12.
- Calls 13-20: verify and correct. Call 20+: no new exploration.
```

```
- Finish with `git status --short && git diff`, then `submit_patch`.

PROCEDURE
1. Extract exact names, messages, types, and edge cases from the issue.
2. Locate the code where the wrong behavior is produced; read one
caller/test.
3. Make the smallest source-only change; match surrounding style.
4. Apply to parallel implementations (sync/async, Client/AsyncClient,
backends, __all__).
5. Verify with at most two targeted test calls or one inline assertion.
6. Clean scratch files; confirm only source files changed; submit.

DECISION RULES
- Ambiguous issue: most literal reading.
- Several locations: where the behavior is produced.
- Search empty: one different string, then edit the best candidate.
- Stuck: different, smaller change. Never conclude nothing needs to change.
```

This is shorter than the public prompt; it may be enough. Measure both.

---

5. Tool policy

5.1 `run_command`

Use it for:

- `git grep -n`, `git ls-files`, `rg -n` for exact symbols;
- `sed -n 'start,endp'` for small reads;
- `git status --short`, `git diff --stat`, `git diff`;
- targeted tests with `timeout`;
- inline Python replacements when `edit_file` fails.

Avoid:

- `cat` large files (5,000-char truncation);
- `find .` full tree (workspace layout already gives 200 entries);
- full `pytest` (time/memory; use one file/`-k`);
- `pip install` (offline; deps preinstalled);
- reading outside `/workspace` (leakage; hidden repos won't have it).

5.2 `read_file`

- Use line ranges; 150 lines/10,000 chars cap.
- Read the implementation plus one caller/test, not whole files.
- If truncated, continue from `end_line + 1`.

5.3 `edit_file`

- Copy `old_string` exactly; choose a short unique match.
- If multiple matches, add surrounding context or use `allow_multiple` only when intended.
- If no match, read the exact region and retry with a smaller snippet.
- After two failures, use one inline Python replace with an assert.
- Keep edits incremental; large `new_string` can truncate the tool call.

5.4 `write_file`

- Only for new source files the issue requires.
- Creates parent dirs; content becomes part of the patch.
- Do not use it to rewrite an existing file from memory; use `edit_file`.

5.5 `get_status` and `submit_patch`

- `get_status` is free; call it when the budget warning appears or before a long verification.
- `submit_patch` is free and ends the loop after the turn; call it last.
- Confirm `patch_size > 0` and `files_changed > 0`; if zero, edit and submit again.

5.6 Graph tools

The host provides precomputed AST call/dependency graphs and 256-d embeddings for the public tasks (S2); hidden tasks use the same pipeline, but S2 does not separately state the hidden embedding dimension (S5:443). They are useful when:

- you know a fully-qualified symbol;
- you need callers/callees quickly;
- the repo is large and text search is noisy.

They are weak when:

- you pass natural-language queries (`search_similar_code` resolves against node keys/suffixes; lexical fallback);
- the symbol is async (missing from graphs; public only, private tasks unaffected; S33);
- the graph files are empty/missing (public audits; S36–S38).

The public prompt-only config declares all nine tools but forbids the three graph tools (S36). That is a reproduction choice, not a recommendation. **Production default: remove the three graph tools from `tools` unless E6 shows a win.** Declaring them costs prompt tokens and adds a rule the model must obey; removing them means the harness never advertises them (S5:282). Run E6 once on 15 tasks with/without; keep them only if they reduce calls or improve localization.

---

6. Context management

6.1 The 32k budget

Every model call sees the conversation history plus tool results. A single `read_file` can return 10,000 chars (~2,500 tokens); three of those plus thoughts can fill a large fraction of the window. The harness compacts at 14,336 tokens, but compaction can lose tool results (S28).

Practical rules:

- Read small regions, not whole files.
- Use `run_command` with `head`/`grep` to reduce output.
- Don’t paste large diffs into the prompt; use `git diff --stat` and targeted hunks.
- Keep the system prompt short.
- Use a read-only sub-agent if exploration is unavoidable; `skip_summarization: true` keeps its outputs out of the root.
- If the model starts repeating calls, the context may be polluted; a fresh sub-agent or a shorter plan can help.

6.2 Compaction behavior

Compaction runs every 15 events, with 2-event overlap, 5-event retention, threshold 14,336 (S5:518-521). It is not a hard reset; it summarizes. The summary may drop tool results and instructions. Do not rely on a rule being remembered for 50 calls. Repeat critical rules in the prompt and in the finalization phase.

6.3 Sub-agent context isolation

`agent_tool` with `skip_summarization: true` is the main escape hatch (S5:626). The sub-agent's intermediate `read_file`/graph outputs stay in its own context; the root gets the summary. Use it for:

- "Find the relevant file/function and return a 5-line summary."
- "Find the tests that cover this function and return paths/line numbers."
- "Trace callers of this symbol and return the call chain."

Do not use it for editing; the root should own the patch.

---

7. Failure modes and prompt fixes

Symptom	Likely cause	Prompt/tool fix
No patch	Agent reads until budget ends	Hard edit by call 12; "pure reading cannot score"
Wrong file	Agent edits where the error is observed, not produced	"Find where the wrong behavior is produced"; read one caller/test
Repeated identical calls	Malformed tool args or loop	"Never repeat an identical failed call"; smaller edits; thinking
Test edits	Agent tries to "fix" tests	Explicit protected paths; "tests are stripped"
Scratch file in patch	Repro script in `/workspace`	"Scratch only in `/tmp`"; final `git status`
Patch too large	Rewriting files	"Smallest unique change"; `edit_file` not `write_file`

Symptom	Likely cause	Prompt/tool fix
Full suite timeout	`pytest` whole repo	“Name one test file; use `-k`; `timeout 150`”
Context overflow	Too much reading	Small ranges; sub-agent; `head`/`grep`
Timeout	Thinking/exploration too high	Lower thinking; earlier edit; per-task cap
Hidden tests fail	Wrong edge cases/messages	Extract exact spec; handle None/empty/unicode/duplicates; exact exception/message
Adapter loops	Alpha/rank too high	Sweep alpha; lower rank; see `03-post-training.md`

8. Local evaluation of the scaffold

Before changing the model, evaluate the scaffold on a **calibrated** local set:

1. Use the public tasks and snapshots (S2).
2. Build a 20–40 task holdout, grouped by repository; don’t tune on all 129. Make one holdout leave-one-repo-out if possible.
3. **E0 failure taxonomy first:** sample 30 failed traces, classify them (no edit, wrong file, wrong edge case, test edit, loop, environment), and convert the top two classes into prompt rules. This is usually a larger gain per hour than a LoRA.
4. Run the public prompt-only config as a baseline.
5. Measure: patch rate, resolve rate, p50/p95 time, setup time, tool calls, turns, malformed calls, repeated-call streaks, compaction events.
6. Change one variable; run paired (same tasks, same seed, alternate order).
7. Promote only if the candidate has ≥8 discordant pairs and ≥75% wins (or two seeds) and does not regress p95/patch rate.
8. Re-check gold patches in the same environment; if gold fails, fix the environment, not the agent.

The public prompt-only config is a strong baseline; a candidate that wins by 1 task on 20 tasks is noise. Use 30–40 tasks for a real comparison, and remember that local-to-hidden transfer is weak (S35, S39).

9. When to add a LoRA

Only after:

- the prompt-only baseline is scored and stable;
- you have a calibrated local evaluator;
- you have ≥300–1,000 resolved trajectories with the same thinking/tool format as inference;
- you have a 10-step training smoke test that fits your GPU;
- you can A/B the adapter on the exact W4A16 base and prove it changes outputs.

The public 1,500-trajectory LoRA experiment did **not** beat the base model; its useful output was the diagnosis path (alpha scaling, template, budget, data) (S25). A LoRA is not a shortcut around a weak scaffold. If the prompt-only agent cannot edit by call 12, fix the prompt first.

See `03-post-training.md` for the full training plan.

---

10. Agent-engineering checklist

- ☐ Single `LlmAgent` with the minimum tools.
- ☐ 31B model, explicit on every agent.
- ☐ `max_output_tokens` 8192, temp 0.2, top_p 0.95; thinking 2048 primary, 4096 A/B.
- ☐ Primary 24 calls / 5 min / 240 s / thinking 2048; 28 calls / 8 min / 4096 as an A/B fallback.
- ☐ Hard edit by call 12.
- ☐ Source-only edits; protected paths listed.
- ☐ Scratch in `/tmp`; final `git status --short && git diff`.
- ☐ `submit_patch` last; confirm `patch_size > 0`.
- ☐ No repeated identical calls; fallback inline replace after two edit failures.
- ☐ No full test suite; one file/`-k`.
- ☐ No reading outside `/workspace`.
- ☐ Graph tools tested, not assumed.
- ☐ Prompt <4k chars; critical rules repeated near the end.
- ☐ Local holdout 20–40 tasks; paired comparisons; gold-patch calibration; leave-one-repo-out holdout if possible.

- ☐ Known-good bundle saved before experimenting.